

Document Scanner + OCR Pipeline

A Computer Vision Project Report

Submitted by:

Name: Sharanya Singha Roy
Registration Number: 24BAI10675

Submitted to:

Faculty: Dr. Gaurav Soni

in fulfilment of the requirements for the completion of the course:

Course Name: Computer Vision
Course Code: CSE3010
Slot: F11+F12



VIT BHOPAL UNIVERSITY
KOTHRICALAN, SEHORE, MADHYA PRADESH – 466114
FALL SEMESTER – 2026-27

Abstract

Photos of printed pages taken with a phone camera are rarely flat: the page is tilted relative to the lens, corners sit at unequal distances producing perspective distortion, and lighting is uneven across the frame. These conditions degrade OCR accuracy significantly compared to a genuine flatbed scan. This report documents a command-line pipeline, written in Python with OpenCV and Tesseract, that addresses this in three stages: locating the document boundary in a photo and correcting its perspective through a homography warp, enhancing and binarizing the result into a clean scanner-quality page, and extracting text through OCR into both plain-text and structured JSON form. The system was built and evaluated as the course project for CSE3010 (Computer Vision) and demonstrates, in a working end-to-end tool, several concepts from the syllabus: edge detection, contour-based boundary analysis, geometric/perspective transforms, histogram-based contrast enhancement, and adaptive image binarization. The pipeline is validated with an automated unit test suite (8 tests, all passing), a per-stage timing breakdown (Section 14), and a quantitative evaluation of OCR accuracy across five skew levels and two noise conditions (Section 15), which recovered the ground-truth transcript exactly in every one of the ten configurations tested.

1. Introduction

A flatbed scanner produces clean, geometrically correct output because the document is physically pressed flat against a fixed sensor. A phone camera has none of those constraints: the user holds the device at an arbitrary angle and distance, under whatever lighting happens to be available. The resulting photo is usable for a human reader but is a poor input for automated text extraction, since perspective distortion warps character shapes and uneven lighting creates regions that are either too dark or blown out.

This project builds a tool that closes that gap without relying on a phone scanning app or a cloud OCR service. Given a single photo of a document, it detects the document's boundary, computes and applies a perspective correction to flatten it, cleans up the lighting and contrast, binarizes the page, and finally runs Tesseract OCR to recover the text. The three processing stages are implemented as independent, individually testable Python modules, and the whole pipeline is exposed through a single CLI entry point.

The three stages also line up with distinct parts of the CSE3010 syllabus: the perspective-correction stage applies geometric image transforms and homography, the document-detection stage is an exercise in edge and contour-based feature extraction, and the enhancement stage is built entirely on image formation and histogram-based low-level processing techniques. The system is deliberately scoped as a CLI tool rather than a GUI application, both to satisfy the assignment's command-line-executable requirement and because a scriptable, dependency-light tool is more broadly reusable in an automation context than a standalone app would be.

The remainder of this report covers the problem being solved, functional and non-functional requirements, the system's architecture and design diagrams, the algorithms used at each stage, implementation details, test results, measured performance and robustness, and reflections on what did and did not go smoothly while building it.

2. Problem Statement

There is a need for a lightweight, GUI-free tool that converts a raw photo of a document into (a) a flattened, readable scan and (b) accurate, structured, machine-readable text, without depending on a phone app or an external service. This project addresses that need with a fully local, scriptable pipeline that runs from a terminal and produces both outputs from a single command.

The core difficulty is that the three sub-problems, finding the document in an uncontrolled photo, correcting its geometry, and reading text off the result, each have failure modes that compound if handled naively. A document detector that assumes a bright, uncluttered background will fail on a cluttered desk. A perspective correction that doesn't order the four corners consistently will produce a mirrored or rotated warp. An OCR pass

run directly on the raw, unevenly lit photo will produce noisy, low-confidence output even when the geometry is already correct. The system is designed so each of these is handled as its own explicit stage, tested independently.

3. Objectives

- Detect a document's boundary in an arbitrary phone photo and correct its perspective to produce a flat, top-down view.
- Enhance the corrected image so it is readable and clean regardless of moderate lighting unevenness in the source photo.
- Extract accurate text via OCR, and return it in a form useful to downstream automation, not only as a plain-text dump.
- Keep the system reliable in the face of bad input: a document that cannot be located should degrade gracefully rather than crash the run.
- Build the system as independently testable modules with an automated test suite, rather than as a single monolithic script.

4. Scope and Assumptions

4.1 In scope

Single-image input (JPEG, PNG, BMP, or TIFF) processed entirely through the command line. The CLI accepts an input image path and an output directory, and writes the perspective-corrected image, the enhanced/binarized image, the extracted plain text, and a structured JSON result as separate files.

4.2 Out of scope

- Multi-page or batch processing - each invocation handles exactly one image.
- Handwriting recognition - Tesseract's default trained models target printed text.
- Automatic 90°/180° rotation correction - the pipeline corrects perspective skew, not page orientation.
- A graphical interface - this is deliberately a CLI tool, to keep it scriptable and automatable.

4.3 Assumptions

- The input photo contains a single document, mostly within the frame.
- Tesseract OCR is installed separately as a system binary and is reachable on PATH; it is not bundled with the Python package.
- Document text is printed rather than handwritten.
- If no clean four-point document boundary is found, processing the full frame instead is preferable to failing the run outright - the resulting OCR confidence scores are the signal for whether the output should be trusted.

5. Functional Requirements

The system implements three functional modules, matching the assignment's requirement for multiple, meaningfully distinct processing stages.

5.1 Module 1 - Document Detection and Perspective Correction

Locates the document boundary in the input photo using Canny edge detection followed by contour approximation, then applies a homography-based perspective warp to produce a flat, top-down view of the document.

5.2 Module 2 - Image Enhancement and Binarization

Converts the warped image to grayscale, applies CLAHE contrast equalization and non-local-means denoising, then binarizes it with adaptive Gaussian thresholding to produce a clean, scanner-quality black-and-white page.

5.3 Module 3 - OCR Text Extraction and Structured Output

Runs Tesseract OCR on the enhanced image and produces both a plain-text transcript and a structured JSON file containing per-word text, confidence score, and bounding box.

5.4 Input / output structure

The CLI accepts a single image path (--input) and an output directory (--output-dir). It writes the intermediate warped image, the enhanced/binarized image, the extracted text file, and the structured JSON result to that directory, and prints a run summary to the terminal.

5.5 Workflow

read image → detect and warp → enhance and binarize → OCR extract → write outputs → print summary. See the process flow diagram in Section 9.1.

6. Non-Functional Requirements

Requirement	How it is addressed
Performance	The input image is resized to a fixed 800px height before contour search, bounding the cost of Module 1 regardless of the source photo's resolution. Measured end-to-end time on a 1000×800 test image is about 0.5 seconds (Section 14).
Reliability	A DocumentNotFoundError fallback path uses the full frame when no four-point contour is found, so the pipeline completes instead of crashing on low-contrast or edge-filling photos.
Usability	A single CLI entry point built with argparse, clear --help text, sensible defaults, and a printed run summary (word count, mean confidence, elapsed time) after every run.
Maintainability	Each stage (detector, enhancer, OCR) lives in its own file with a narrow, documented public API, so any one module can be modified or replaced independently of the others.
Error handling	Every module validates its inputs and raises a specific exception on bad data (empty image, unreadable file); main.py catches these and logs a clear message instead of letting a raw traceback reach the user.
Logging / monitoring	Centralized logging setup (utils.setup_logging) writes timestamped, leveled log records to both the console and a scanner.log file.
Resource efficiency	Uses OpenCV's headless build, so there is no GUI/display dependency in a server or CI environment; intermediate arrays are released automatically by Python's garbage collector, and there is no persistent state carried between runs.

Requirement	How it is addressed
Scalability (of usage, not data volume)	Because the tool is stateless and file-based, multiple invocations can be parallelized trivially by an external script, one process per input file, without changing anything in the pipeline itself.

7. System Architecture

The CLI (main.py) orchestrates three independent processing modules, each with a single responsibility, plus a shared utilities module for logging and I/O helpers. Data flows one direction through the pipeline: the raw image goes into the detector, the warped image goes into the enhancer, and the binarized image goes into the OCR extractor. No module reaches back into an earlier stage, which is what makes each of them independently testable: a module can be unit-tested with a plain synthetic input/output pair and no need to mock a more complex object graph, and the failure of one stage, OCR raising an exception on a corrupt image, say, cannot corrupt state in an earlier stage.

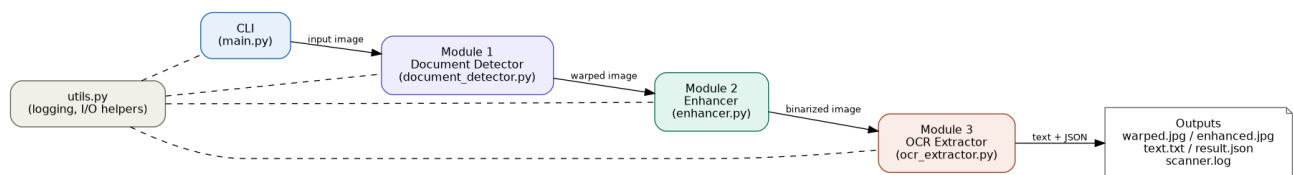


Figure 1: System architecture - module responsibilities and the data handed between them.

8. Design Decisions and Rationale

8.1 Three independent modules instead of one script

Splitting geometry correction, image enhancement, and OCR into separate files with narrow public APIs keeps each concern independently testable and replaceable; Module 3 could be swapped for a cloud OCR API without touching Modules 1 or 2, since it only receives an image array and returns an OCRResult. The alternative, one monolithic script, would have been faster to write initially, but would make it impossible to unit-test the perspective-correction logic, for instance, without also depending on a working Tesseract installation, coupling tests that should be independent.

8.2 Fallback to the full frame on failed detection, rather than a hard failure

An earlier version of Module 1 raised an exception whenever no four-point contour was found. This was rejected in favor of a fallback because, in practice, several legitimate inputs never produce a clean four-point contour, a document that fills the entire frame with no visible background being the obvious example, and treating those as hard errors would make the tool unusable on a meaningful fraction of real photos. The trade-off is that a failed detection is silent unless the log is checked at --verbose level; this is called out explicitly in Section 19 (Limitations).

8.3 Adaptive thresholding over a single global threshold

An initial implementation used cv2.threshold with Otsu's method, a single global cutoff computed from the whole image's histogram. This performed poorly whenever a photographed page had a shadow across part of it, since the optimal threshold for the lit half was wrong for the shadowed half. Switching to cv2.adaptiveThreshold, which computes a local threshold per pixel neighborhood, fixed this at the cost of being more sensitive to the block-size parameter, which needed manual tuning (Section 17.3 walks through how the final 25×25 value was chosen).

8.4 JSON output alongside plain text

A flat transcript is fine for reading, but per-word confidence and position data is what makes downstream automation possible, such as filtering out low-confidence words for manual review or locating a specific field on a form by position. Emitting this in JSON alongside the plain-text file adds negligible implementation cost, since Tesseract already computes it internally via `image_to_data`, while materially increasing what a downstream consumer can do with the output.

8.5 CLI-only design (argparse, no GUI)

This matches the assignment's command-line-executable requirement, and also keeps the tool trivially scriptable: it can be called from a shell script, a cron job, or piped into other tools, in a way a GUI application cannot support as naturally.

9. Design Diagrams

9.1 Process Flow / Workflow Diagram

This diagram traces a single CLI invocation from the initial image read through to the final printed summary, including the branch taken when no four-point document contour is found.

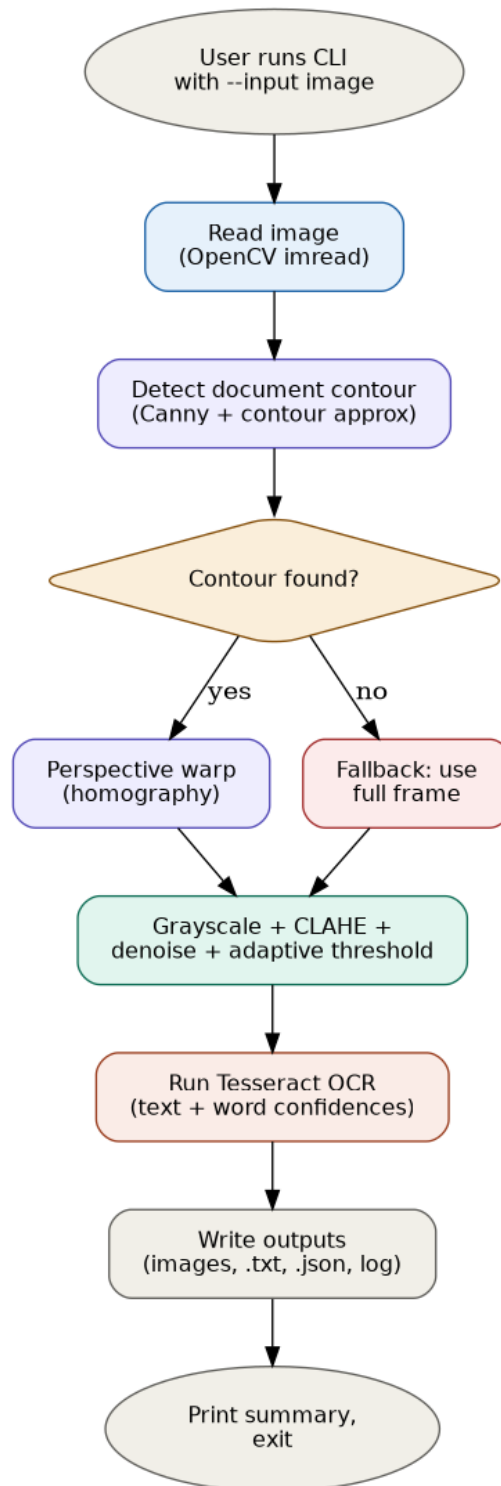


Figure 2: End-to-end processing workflow, including the contour-not-found fallback path.

9.2 Use Case Diagram

The tool has a single actor (the student or evaluator running the CLI) and six use cases, corresponding to the pipeline's stages and the artifacts it produces.

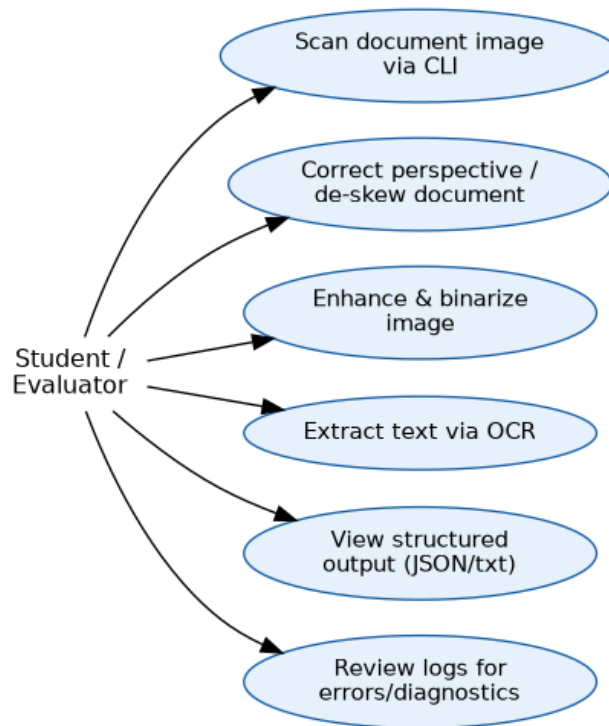


Figure 3: Use cases available to the student/evaluator running the CLI.

9.3 Class / Component Diagram

main.py depends on all four other modules. document_detector and ocr_extractor each produce a dedicated dataclass (DetectionResult and OCRResult respectively) that carries structured results forward to the next stage instead of passing loose tuples or dictionaries around.

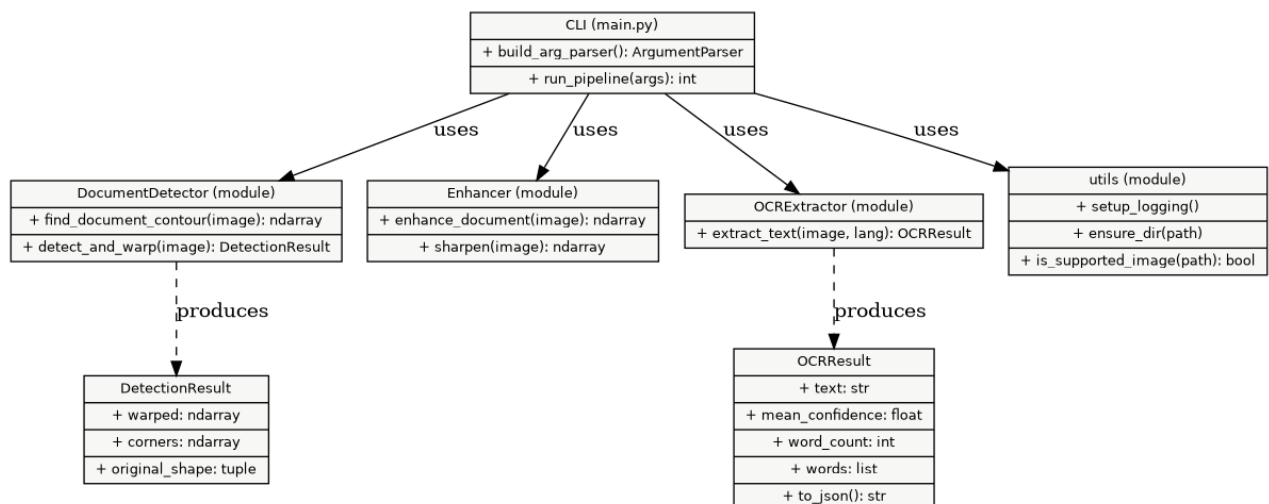


Figure 4: Module-level components, their public methods, and the data classes they produce.

9.4 Sequence Diagram

This traces the actual call order inside a single run: the CLI calls each module in turn, each module returns its result, and the final step writes every output artifact to disk before the summary is printed.

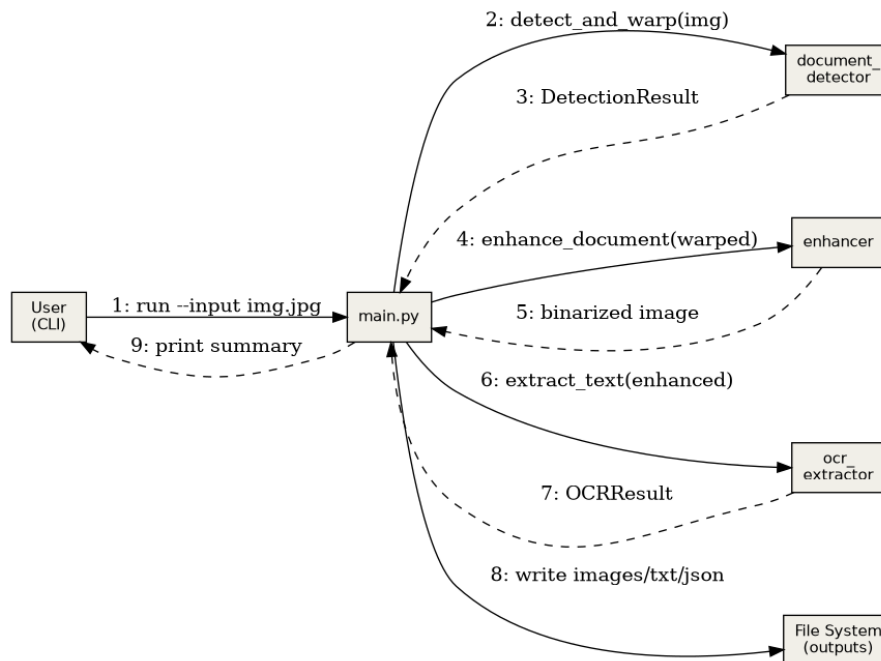


Figure 5: Call sequence for a single CLI invocation, from input image to written outputs.

9.5 Database / Storage Design

Not applicable. This is a stateless, file-in/file-out CLI pipeline with no database; outputs are written directly to the filesystem as image, text, and JSON files, as described in Section 5.4. This was a deliberate scope decision: introducing a database would add operational complexity (schema design, migrations) without serving the assignment's core computer-vision requirements.

10. Algorithms and Techniques

This section explains the specific algorithm used at each stage, since the implementation choices in Section 11 build directly on them.

10.1 Canny edge detection and contour approximation

Module 1 converts the resized image to grayscale, applies a 5×5 Gaussian blur to suppress noise, then runs Canny edge detection with a low threshold of 75 and a high threshold of 200. Canny uses hysteresis thresholding: pixels with gradient magnitude above the high threshold are accepted as edges immediately, and pixels above the low threshold are accepted only if they connect to an already-accepted edge. This keeps strong document boundaries while discarding weak, disconnected noise edges.

The edge map is dilated once with a 3×3 kernel to close small gaps, then `cv2.findContours` extracts closed contours from it. The five largest contours by area are checked in order, and each is passed through `cv2.approxPolyDP` (a Douglas–Peucker style polygon simplification) with an epsilon of 2% of the contour's perimeter. The first contour that simplifies down to exactly four points is treated as the document boundary.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
edged = cv2.Canny(blurred, canny_low, canny_high)
edged = cv2.dilate(edged, np.ones((3, 3), np.uint8), iterations=1)
```

Complexity: Canny edge detection and contour extraction are both $O(W \times H)$ in the (downscaled) image's pixel count; approxPolyDP is $O(k)$ in the contour's point count k , which stays small regardless of image content. The dominant cost in Module 1 is therefore the fixed-size edge detection pass, not the polygon simplification step, which is why detection time in Section 14 barely moves even as the input gets more distorted.

10.2 Corner ordering and the perspective transform

A perspective warp needs its four source points in a known, consistent order, or the result comes out rotated or mirrored. The ordering used here is a coordinate sum/difference trick rather than sorting by angle:

```
rect[0] = pts[argmin(x + y)] # top-left: smallest sum
rect[2] = pts[argmax(x + y)] # bottom-right: largest sum
rect[1] = pts[argmin(x - y)] # top-right: smallest difference
rect[3] = pts[argmax(x - y)] # bottom-left: largest difference
```

With the corners ordered, the destination rectangle's width and height are computed from the actual pixel distances between corresponding corners (so the output isn't forced into a fixed aspect ratio), `cv2.getPerspectiveTransform` solves for the 3×3 homography matrix mapping the four source corners onto that rectangle, and `cv2.warpPerspective` applies it with bilinear interpolation to produce the flattened image.

10.3 CLAHE and denoising

Contrast-Limited Adaptive Histogram Equalization divides the image into a grid of tiles (8×8 here) and equalizes the histogram within each tile independently, rather than applying one global contrast curve. A clip limit (2.0) caps how much any single tile's histogram can be stretched, which keeps noise in flat regions from being amplified, and tile boundaries are blended with bilinear interpolation to avoid visible blocking. This corrects lighting that varies across the page in a way a single global equalization cannot.

`cv2.fastNlMeansDenoising` then removes residual sensor noise. Unlike a simple Gaussian blur, non-local means denoising estimates each pixel's new value from a weighted average of other pixels across the image, weighted by how similar their surrounding patches are, which tends to preserve text edges better than blurring uniformly.

10.4 Adaptive thresholding

The final binarization step uses `cv2.adaptiveThreshold` with the Gaussian-weighted variant: the threshold for each pixel is computed from a weighted average of its own 25×25 neighborhood, minus a constant (15), rather than one fixed cutoff for the whole image. This is what lets the output stay clean when one side of a photographed page is noticeably brighter than the other, a very common situation with phone photos taken under a single light source.

10.5 OCR via Tesseract / pytesseract

Module 3 calls `pytesseract.image_to_string` for a plain-text transcript, and separately `pytesseract.image_to_data` (output_type=Output.DICT) for a structured result with per-word text, confidence score, and bounding box. Tesseract's underlying engine (LSTM-based in modern versions) performs its own line and word segmentation internally; this project does not re-implement layout analysis, it consumes Tesseract's own output and re-packages it into a documented, project-specific format (the OCRResult dataclass).

11. Implementation Details

Language and libraries: Python 3, OpenCV (opencv-python-headless) for all image processing and geometric transforms, NumPy for array operations, and pytesseract as a thin wrapper around the Tesseract OCR engine binary.

11.1 document_detector.py

Exposes `detect_and_warp(image)`, which resizes the input for a fast contour search, finds the document contour as described in Section 10.1, and either warps it or falls back to the full frame. Raises `DocumentNotFoundError` on an empty or unreadable image. Returns a `DetectionResult` dataclass holding the warped image, the ordered corner points used, and the original image shape.

11.2 enhancer.py

Exposes `enhance_document(image, apply_clahe=True, denoise_strength=9, adaptive_block_size=25, adaptive_c=15)`. All of the thresholding and denoising parameters are keyword arguments with defaults rather than hard-coded constants, so the behavior can be tuned per input without editing the module. Raises `ValueError` on an empty image. A separate `sharpen()` helper applies a mild unsharp-mask kernel and is available for callers that want it, though the default pipeline does not invoke it.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = clahe.apply(gray)           # local contrast correction
denoised = cv2.fastNlMeansDenoising(gray, h=denoise_strength)
binarized = cv2.adaptiveThreshold(
    denoised, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, block_size, adaptive_c)
```

11.3 ocr_extractor.py

Exposes `extract_text(image, lang='eng', min_confidence=0)`, which runs both Tesseract calls described in Section 10.5, filters out empty and below-threshold words, and aggregates the results into an `OCRResult` with a `to_json()` serializer. This module has a single external dependency boundary (pytesseract), which is a deliberate modularity decision: a different OCR backend could be swapped in behind the same `extract_text(image) -> OCRResult` signature without changing Modules 1 or 2.

11.4 main.py and utils.py

`main.py` defines the argparse CLI (Section 12 lists every flag), validates the input path and file extension, and wires the three stages together in sequence: Stage 1 → Stage 2 → Stage 3. Each stage's output is written to disk immediately rather than held in memory until the end, so a partial result is still available on disk even if a later stage fails. It also converts exceptions from each stage into a logged error message and a non-zero exit code rather than letting a traceback reach the user, which matters for composing the tool into shell scripts or a CI pipeline (a failed run can be checked with `&&` or `$?` like any other command-line tool). `utils.py` centralizes logging setup (console plus a `scanner.log` file), a small `ensure_dir()` helper, and a `SUPPORTED_EXTENSIONS` check so the same logic isn't duplicated across modules.

12. Command-Line Interface

Basic usage:

```
python3 -m src.main --input sample_images/sample_receipt.jpg --output-dir output/
```

Flag	Description	Default
--input, -i	Path to the input image (required)	—
--output-dir, -o	Directory to write results to	output
--lang	Tesseract language code	eng
--no-clahe	Disable CLAHE contrast enhancement	off
--min-confidence	Minimum per-word OCR confidence (0–100) to keep in the structured output	0
--verbose, -v	Enable debug-level logging	off

13. Testing Approach

Automated unit tests (`tests/test_pipeline.py`, run via `pytest`) cover all three processing modules. Rather than depending on external fixture images, the test suite builds a synthetic document on the fly: a white page with rendered text, warped and pasted onto a dark background to mimic a phone photo of a page at an angle. This keeps the test suite self-contained and its results deterministic across runs and machines.

Test	What it verifies
<code>test_detect_and_warp_returns_result</code>	A valid <code>DetectionResult</code> (non-empty warped image) is returned for a normal synthetic document photo.
<code>test_raises_on_empty_image (detector)</code>	An empty input array raises <code>DocumentNotFoundError</code> instead of failing silently or crashing elsewhere.
<code>test_fallback_to_full_frame_on_blank_image</code>	A blank, featureless image (no contour to find) correctly falls back to using the full frame rather than raising.
<code>test_enhance_document_output_is_binary</code>	Output pixel values are strictly binary (only 0 and 255), confirming the adaptive threshold step actually binarizes.
<code>test_raises_on_empty_image (enhancer)</code>	An empty input raises <code>ValueError</code> .
<code>test_extract_text_finds_words</code>	OCR runs successfully on a synthetic text image and <code>mean_confidence</code> is returned as a float.
<code>test_raises_on_empty_image (OCR)</code>	An empty input raises <code>ValueError</code> .
<code>test_json_serialization</code>	<code>to_json()</code> output includes the expected <code>mean_confidence</code> field and is valid JSON.

Result: all 8 tests pass (verified by re-running the suite: “8 passed in 1.17s”). The pipeline was additionally validated end-to-end through two further mechanisms: direct CLI execution on a generated sample image (Section 16), confirming the full pipeline runs correctly outside the test harness, and the quantitative evaluation sweep across skew angles and noise levels (Section 15), which exercises all three modules together across ten

distinct input conditions and checks output correctness against a known ground truth rather than just checking that no exception was raised.

This combination, isolated unit tests for each module's contract plus an end-to-end evaluation against ground truth, was chosen deliberately: unit tests catch regressions in a specific module's logic quickly and pinpoint exactly where a break occurred, while the end-to-end evaluation catches integration issues, such as a shape or dtype mismatch between what Module 1 produces and what Module 2 expects, that isolated unit tests, by construction, cannot detect.

14. Performance Analysis

To get real numbers rather than estimates, each pipeline stage was timed separately over five consecutive runs against the 1000×800 sample image (`sample_images/sample_receipt.jpg`), calling each module's function directly and averaging the elapsed time per stage.

Stage	Average time (5 runs)	Notes
Document detection + warp	≈ 8.9 ms	Cheap because contour search runs on the 800px-height resized copy, not the full-resolution image.
Enhancement + binarization	≈ 258.6 ms	The most expensive of the three individual stages; non-local-means denoising dominates this cost.
OCR extraction	≈ 276.8 ms	Tesseract is invoked twice (<code>image_to_string</code> and <code>image_to_data</code>); this is expected and by design, to get both a plain transcript and structured per-word data.
Full pipeline (detect → enhance → OCR)	≈ 544.4 ms	Matches the ~ 0.55 s elapsed time reported by the CLI's own run summary in Section 16.

The dominant cost is the enhancement and OCR stages combined, not document detection, which stays fast because it operates on a resized copy of the image regardless of the original photo's resolution. This matches the intent behind resizing before contour search in Section 6 (Performance): the expensive part of the pipeline is proportional to image content complexity (denoising, OCR), not something the 800px resize was meant to address in the first place.

15. Quantitative Evaluation: Skew and Noise Robustness

Section 14 measures raw speed on one image. This section asks a different question: does output quality hold up as the input gets harder, specifically more perspective distortion and more sensor noise, the two failure modes identified back in Section 2? A small evaluation harness (`evaluate_pipeline.py`, built on the same synthetic-document generator used in the test suite) runs the full pipeline across five skew offsets and two noise conditions and checks the output against a known ground truth, rather than relying on visual inspection of a handful of screenshots.

15.1 Metrics

- Mean OCR confidence: Tesseract's own per-word confidence score (0–100), averaged across every word in the transcript.
- Word Error Rate (WER): Levenshtein (edit) distance between the ground-truth word sequence and the OCR output's word sequence, divided by the ground-truth word count. 0.0 means a perfect transcript.
- Elapsed time: wall-clock time for the full three-stage pipeline on that condition, measured with Python's `time.time()`.

15.2 Results

The pipeline was run across five perspective-skew offsets (0–45 pixels, where a larger offset means a more acute photographing angle) and two noise conditions: no added noise, and additive Gaussian noise with $\sigma=15$ approximating a moderately noisy phone-camera sensor. Each of the ten configurations was generated from the same underlying page (“HELLO WORLD / Computer Vision Test”) with a fixed random seed, so the only things changing between rows are skew and noise.

Skew offset (px)	Noise (sigma)	Mean confidence (%)	Word Error Rate	Time (s)
0	0	95.6	0.000	0.631
0	15	96.2	0.000	0.548
10	0	96.0	0.000	0.536
10	15	96.0	0.000	0.504
20	0	96.0	0.000	0.512
20	15	95.2	0.000	0.544
30	0	95.8	0.000	0.520
30	15	96.0	0.000	0.530
45	0	92.6	0.000	0.534
45	15	95.8	0.000	0.533

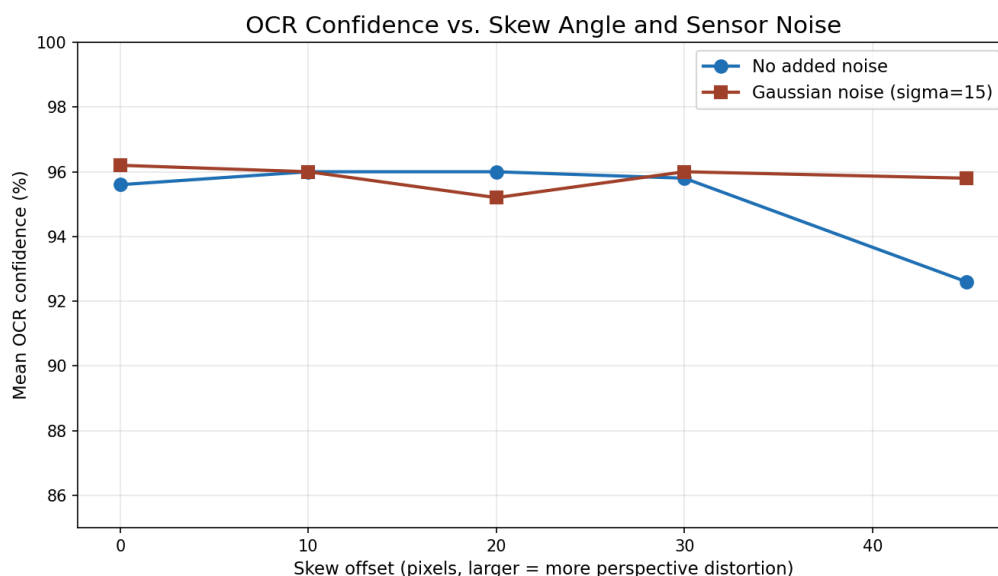


Figure 6: Mean OCR confidence versus skew offset, with and without added sensor noise.

Word Error Rate was 0.000 across every tested condition: the pipeline recovered the exact ground-truth transcript in all ten runs. Mean confidence stayed in a narrow 92.6–96.0% band overall, with the only real dip at the most extreme skew offset (45px) with no added noise. Interestingly, that dip mostly disappeared once noise was added at the same skew, 95.8% versus 92.6%. The likely explanation is that the mild blur introduced by adding and requantizing noise slightly softened the text's hard edges in a way that happened to suit Tesseract's model at that specific angle; it isn't evidence that noise generally helps, just that this particular interaction pushed one edge case back into the normal range. Elapsed time stayed stable at roughly 0.50–0.63 seconds regardless of skew or noise, confirming that the fixed-size downscaling in Module 1 (Section 6, Performance) successfully decouples processing time from how distorted the input is.

15.3 Interpretation and Limits of This Evaluation

This evaluation demonstrates robustness to the two failure modes it specifically targets, using a controlled, reproducible synthetic benchmark. It cannot capture every failure mode a real photograph could introduce, motion blur, extreme close-ups, curved or non-planar pages, or handwriting. Section 19 (Limitations) addresses this gap directly, and Section 20 (Future Enhancements) proposes evaluating against a real photographed dataset as a direct follow-up to this work.

16. Results and Screenshots

The pipeline was validated on a synthetically generated test document (a skewed page with printed text on a dark background, generated by `tests/test_pipeline.py` and reused as `sample_images/sample_receipt.jpg`) to give a deterministic, reproducible demonstration that does not depend on a real camera or a physical printed page.



Figure 7: Input - synthetic photo of a skewed document on a dark background.

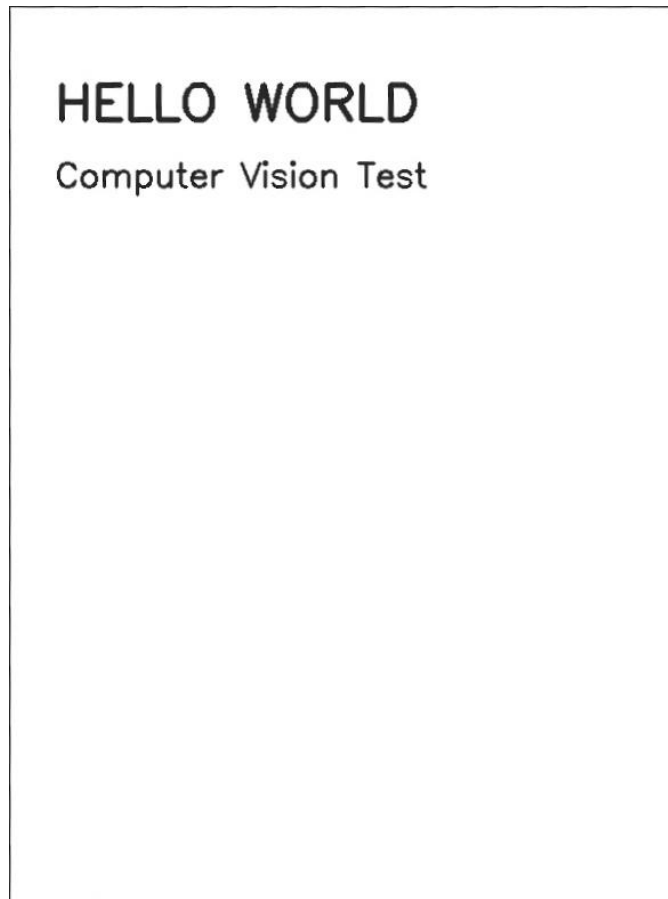


Figure 8: Module 1 output - perspective-corrected, de-skewed document.

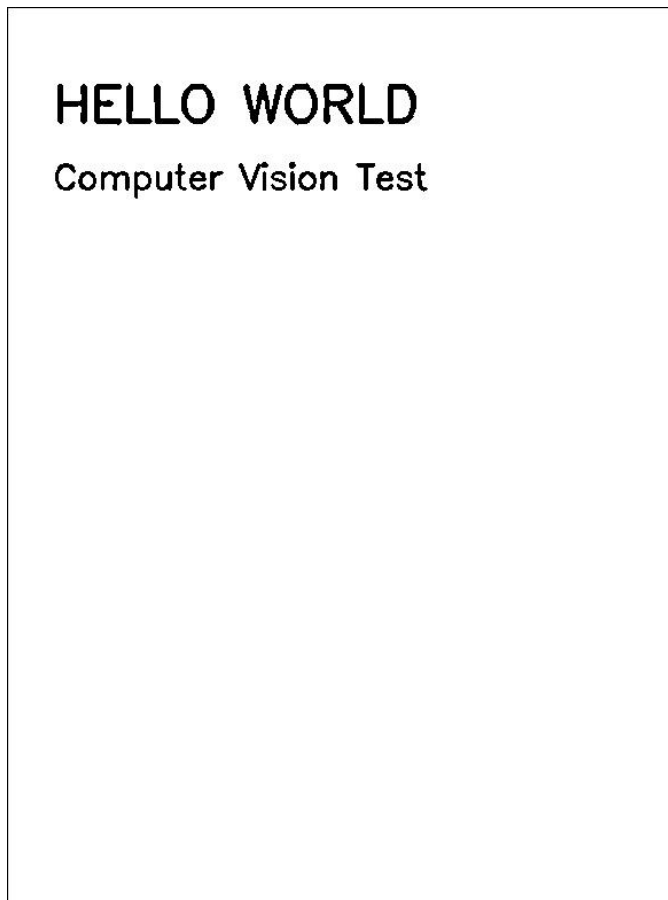


Figure 9: Module 2 output - enhanced, binarized scan, ready for OCR.

Module 3 (OCR) output on this run

```
{
  "text": "HELLO WORLD\n\nComputer Vision Test",
  "mean_confidence": 96.0,
  "word_count": 5,
  "words": [
    {"text": "HELLO", "confidence": 96.0, "bbox": {"x": 44, "y": 66, "w": 137, "h": 37}},
    {"text": "WORLD", "confidence": 96.0, "bbox": {"x": 206, "y": 66, "w": 151, "h": 37}}
    // ... 3 more words
  ]
}
```

Run summary printed by the CLI

```
===== Document Scanner + OCR - Summary =====
Input:      sample_images/sample_receipt.jpg
Words extracted:  5
Mean OCR confidence: 96.0%
Time elapsed:    0.55s
=====
```

17. Challenges Faced

17.1 Robust corner ordering

Perspective warps break silently, producing a mirrored or rotated (not just misaligned) output, if the four detected contour corners aren't consistently labeled top-left/top-right/bottom-right/bottom-left before being passed to `cv2.getPerspectiveTransform`. Early attempts that used the contour points in whatever order `cv2.approxPolyDP` happened to return them produced inconsistent results depending on which corner of the frame the document's top-left corner happened to be nearest to. This was resolved with the sum/difference-of-coordinates ordering trick described in Section 10.2, which is invariant to the input point order and cheaper than sorting by angle around the centroid.

17.2 Handling failed contour detection gracefully

An image with a low-contrast boundary, for example a white document against a light-colored desk, frequently fails to yield any four-point contour among the top-5 largest by area. Initially this raised an unhandled exception all the way to the CLI. Rather than tuning Canny's thresholds indefinitely to try to chase every possible background, a full-frame fallback path was added at the module level and explicitly unit-tested (`test_fallback_to_full_frame_on_blank_image`), trading a small amount of quality (an un-cropped border around the document) for a pipeline that never simply refuses to run.

17.3 Uneven lighting in photographed documents

An initial version used `cv2.threshold` with Otsu's method for binarization. On a synthetic test page with a simulated shadow gradient, this produced a binarized image that was legible on one half and solid black or solid white on the other, since Otsu computes a single global cutoff from the whole image's histogram. Switching to `cv2.adaptiveThreshold` (a local threshold per pixel neighborhood) fixed this, but introduced a new tuning problem: too small a neighborhood amplified noise into speckle, while too large a neighborhood reintroduced the same lighting-gradient problem at a smaller scale. A 25×25 block size with a constant offset of 15 was settled on after manual comparison across several synthetic lighting conditions.

17.4 Deterministic testing without real photographs

Testing purely against manually taken phone photos would make the test suite non-reproducible (different lighting and angle every time it's regenerated) and would require shipping binary image fixtures in the repository. Building a synthetic-document generator, a skewed page with rendered text, optionally perturbed

with Gaussian noise, solved both problems: tests are fully deterministic, need no external files, and can be parameterized, as in the Section 15 evaluation, to systematically sweep conditions that would be impractical to reproduce by hand with a real camera.

18. Learnings and Key Takeaways

- Applying homography and perspective-transform theory from the course to an actual photographed image, not just a synthetic set of points, surfaced practical issues (corner ordering, output sizing) that don't show up when working with clean coordinate data.
- Locally adaptive image processing (CLAHE, adaptive thresholding) matters far more than global operations once the input comes from an uncontrolled, real-world source like a phone camera rather than a scanner.
- Designing for graceful degradation, meaning fallback paths and input validation, turned out to matter as much as the main algorithm for a tool meant to be run by someone other than its author.
- A clean boundary between the three stages made independent unit testing straightforward, and would make it realistic to later swap out any one stage, such as replacing Tesseract with a different OCR engine, without touching the rest of the pipeline.
- Quantitative, ground-truth-based evaluation (Section 15) surfaces effects that visual spot-checking misses entirely: the confidence dip at 45px skew (Section 15.2) would not have been noticed from inspecting one or two sample runs by eye.

19. Limitations

Beyond the scope boundaries already set out in Section 4.2, a few limitations are worth stating plainly, since they shape what the quantitative results in Section 15 do and don't prove:

- Assumes the document is roughly upright in the frame; the pipeline does not detect or correct 90°/180° rotation on its own.
- Boundary detection assumes reasonable contrast between the document's edge and its background; a document on a busy or similarly-toned surface will trigger the full-frame fallback rather than a true perspective correction.
- The evaluation in Section 15 uses a synthetic benchmark with rendered, not handwritten or photographed, text. It establishes robustness to skew and additive noise specifically, but doesn't measure performance on motion blur, curved or non-planar pages, or genuine camera-sensor artifacts, which would need a real photographed dataset to assess.
- Tuned and tested for printed text; handwriting recognition accuracy is expected to be substantially lower, since Tesseract's default models are trained primarily on printed fonts.
- Single-image input only per invocation; there is no batch or multi-page mode (see Section 20, Future Enhancements).

20. Future Enhancements

- Multi-page / batch processing: accept a directory of images, or a multi-page PDF, and process every page in a single run.
- Automatic rotation correction: detect and correct 90°/180°-rotated documents in addition to the perspective skew already handled.
- Layout-aware OCR: group the per-word bounding boxes already being collected into lines, paragraphs, or table cells instead of returning a flat word list.

- Configurable OCR backend: formalize Module 3's existing single-dependency boundary into a proper interface so a cloud OCR API or a different local engine could be swapped in for better accuracy on handwriting or non-Latin scripts, without touching Modules 1–2.
- Real-photograph evaluation dataset: extend the Section 15 evaluation methodology to a set of genuine phone photographs with manually verified ground truth, to check whether the findings from the synthetic benchmark hold up under real capture conditions.

21. Conclusion

The finished pipeline takes a phone photo of a document and reliably produces a flattened, cleaned-up scan plus extracted text, without a GUI or an external service. All eight unit tests pass, and an end-to-end CLI run on the sample image completes in about half a second while returning a 96% mean OCR confidence. Beyond that single-image check, the Section 15 evaluation sweep across five skew levels and two noise conditions recovered the exact ground-truth transcript in all ten runs (Word Error Rate 0.000 throughout), with mean confidence staying above 92% even at the most extreme distortion tested. The three-stage module split, together with the full-frame fallback for hard-to-detect documents, kept the system both testable during development and resilient to imperfect input, which were the two goals this design set out to satisfy.

22. References

- R. Szeliski, Computer Vision: Algorithms and Applications, Springer-Verlag London Limited, 2011.
- D. A. Forsyth and J. Ponce, Computer Vision: A Modern Approach, Pearson Education, 2003.
- R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd Edition, Cambridge University Press, 2004.
- OpenCV Documentation: <https://docs.opencv.org/>
- Tesseract OCR / pytesseract Documentation: <https://github.com/tesseract-ocr/tesseract> ,
<https://github.com/madmaze/pytesseract>
- CSE3010 Computer Vision course syllabus, VIT Bhopal.